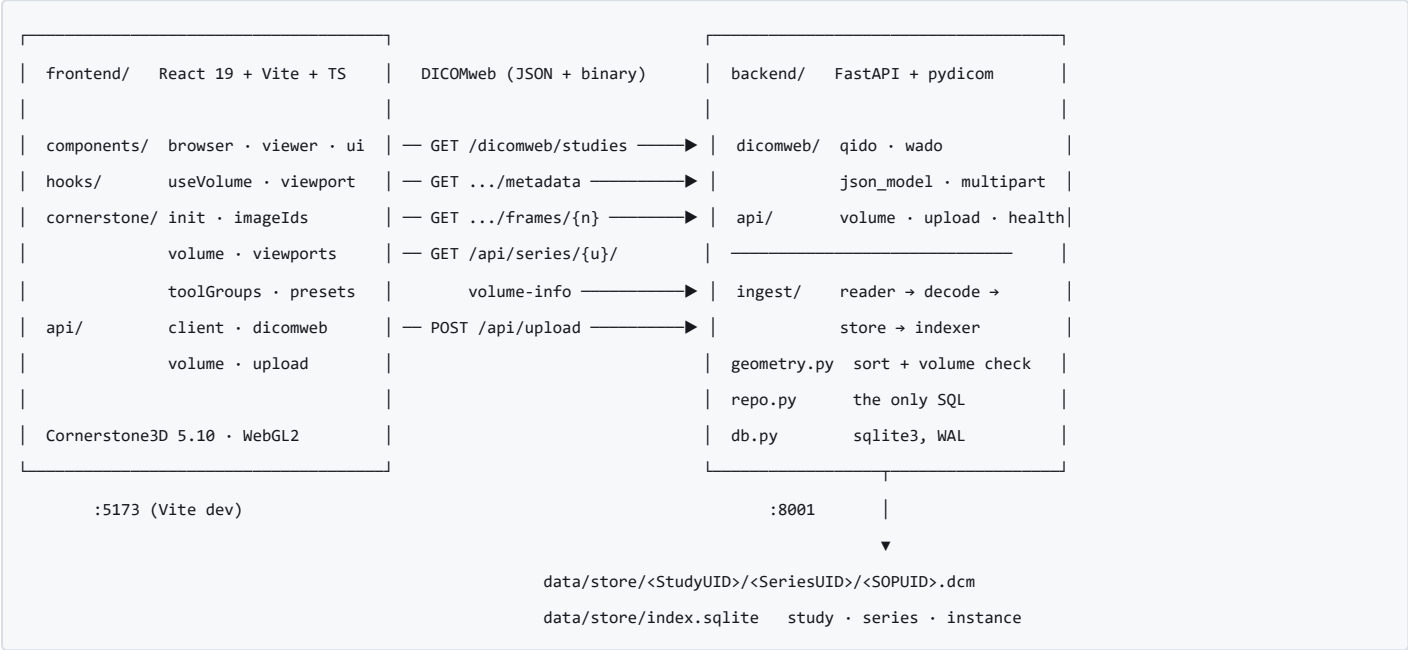


Architecture — DICOM 3D Brain Viewer

Two processes, one repository, no third service. The browser does the rendering; Python does everything that requires understanding DICOM.



Backend — layers, innermost last

Layer	Modules	Responsibility
HTTP	<code>dicomweb/qido.py</code> , <code>dicomweb/wado.py</code> , <code>api/*</code>	Routing, status codes, media types. No SQL, no pixel logic.
Serialisation	<code>dicomweb/json_model.py</code> , <code>dicomweb/multipart.py</code>	DICOM JSON model (PS3.18 F), <code>multipart/related</code> framing.
Ingest	<code>ingest/reader → decode → store → indexer</code>	Validate → decode to Explicit VR LE → file by UID → index → finalize.
Domain	<code>geometry.py</code>	Slice ordering and 3D-grid validation. The only imaging maths.
Persistence	<code>repo.py</code> , <code>db.py</code> , <code>models.py</code>	Every SQL statement lives in <code>repo.py</code> .

Why decode at ingest, not on read. The volume loader pulls every frame of a series up front. Decoding JPEG 2000 once at ingest turns `frames/{n}` into a byte slice of an already-uncompressed file, so frame retrieval never touches a codec.

Connection per request. `db.connect()` opens SQLite with WAL and `busy_timeout`; the `get_db` dependency hands each request its own connection. A single shared connection failed intermittently under the volume loader's parallel frame fetches — `sqlite3.Connection` is not safe across threads.

Frontend — layers, outermost first

Layer	Modules	Responsibility
Routes	<code>App.tsx</code> , <code>BrowserPage</code> , <code>ViewerPage</code>	<code>/</code> and <code>/viewer/:studyUid/:seriesUid</code> .
Components	<code>components/browser/*</code> , <code>components/viewer/*</code>	DOM and React state only.
Hooks	<code>useVolume</code> , <code>useSeriesMetadata</code> , <code>useViewportState</code>	Orchestration: the load state machine, viewport event → React state.
Adapter	<code>cornerstone/*</code>	The only place that touches Cornerstone globals.
Transport	<code>api/*</code>	<code>fetch</code> , DICOM JSON → flat types. No DICOM parsing.

One volume, four viewports. `createAndCacheVolume` builds a single volume in GPU memory; `setVolumesForViewports` attaches it to three `ORTHOGRAPHIC` viewports (axial/sagittal/coronal) and one `VOLUME_3D`. Nothing is loaded twice.

The adapter boundary exists for teardown. Cornerstone keeps global state — rendering engines, tool groups, a volume cache. Concentrating it in `cornerstone/` makes "open a series, go back, open another" a single explicit cleanup path rather than a scatter of `useEffect` returns.

Decisions worth knowing

- **DICOMweb rather than a custom API.** The frontend is a stock Cornerstone3D client; an OHIF instance could point at this backend unmodified.
- **Volume validation is a gate, not a warning.** A series that is not a regular 3D grid cannot be opened; the browser greys it out with the geometric reason.
- **Backend is modality-agnostic.** It validates geometry, not anatomy. The brain-only scope lives in the sample manifest and the preset lists.
- **Port 8001**, because `iphlpvc` commonly holds 8000 on Windows.
- **events aliased to the userland polyfill** in `vite.config.ts`: `vtk.js` pulls in `xmlbuilder2`, whose `XMLBuilderCImpl` extends `EventEmitter`, and Vite stubs bare Node builtins as empty modules for the browser.